

Etape 1 : Créer Projet

Composer create-project laravel/laravel projectName

Etape 2 : Créer les tables en utilisant les migrations

Php artisan make : migration create_tableName_table

Migration : classe contenant deux méthodes up et down , permettant de se connecter avec la base de données sans passer par sql . L'objectif il est soit ajouter une table ou modifier une table existante.

Les commandes artisan associées à la migration :

Php artisan migrate : pour invoquer up(lancer up pour créer ou modifier une table)

Php artisan migrate : rollback => pour annuler la dernière migration

....

Etape 3 : Alimenter les tables en utilisant les factories et les seeders

Exemple 1 : créer factory CatégorieFactory

Commande	Php artisan make :factory CatégorieFactory
Contenu	<pre>public function definition(): array { return [// 'name' => fake()->unique()->word(),]; }</pre>

Exemple 2 : créer factory ProductFactory

Commande	Php artisan make :factory ProductFactory
Contenu	<pre> public function definition(): array { return [// 'prix'=>fake()->randomFloat(2,5,100), 'libelle'=>fake()->unique()->word(); 'categorie_id'=>Categorie::inRandomOrder->first()->id,]; } </pre>

NB. L'objet des factories est : créer des fake data , pour insérer ces données dans la base de données , on doit définir des seeders

Exemple 1 : créer et définir le seeder CategorieSeeder

Commande	Php artisan make :seeder CategorieSeeder
Contenu	<pre> public function run(): void { // Categorie::factory()->count(10)->create(); } </pre>

Exemple 2 : créer et définir le seeder ProductSeeder

Commande	Php artisan make :seeder ProductSeeder
Contenu	<pre> public function run(): void { // Product::factory()->count(20)->create(); } </pre>

Etape 4 : appeler tous les seeders dans la méthode run du main seeder DataBaseSeeder

```

public function run(): void
{
    $this->call([
        CategorySeeder::class,
        ProductSeeder::class
    ]);
}

```

NB. Pour invoquer la méthode run du DatabaseSeeder on exécute :

php artisan db :seed

Etape 5 : Définir les contrôleurs et leurs actions

Contrôleur de base : sans aucune action	Php artisan make :controller CategoryController
Contrôleur de ressource : avec des prototypes d'actions	Php artisan make :controller CategoryController --resource

Exemple : CategoryController

```

<?php

namespace App\Http\Controllers;

use App\Http\Controllers\Controller;
use Illuminate\Http\Request;
use app\Models\Categorie;

class CategoryController extends Controller
{
    /**
     * Display a listing of the resource.
     */
    public function index()
    {
        //
        $categories=Categorie::all();
        return view('index',['categories'=>$categories]);
        //return view('index',compact('categories'));
    }

    /**
     * Show the form for creating a new resource.

```

```

*/
public function create()
{
    //
    return view('create');
}

/**
 * Store a newly created resource in storage.
 */
public function store(Request $request)
{
    //dd($request)
    //store => après le clic sur submit
    //validation
    $request->validate([
        'nom'=>'required|unique:categories|max:255',
    ]);
    Categorie::create($request->all());
    //rediriger l user à la vue index
    return view('index')->with('success','catégorie bien crée !!!');
}

/**
 * Display the specified resource.
 */
public function show(string $id)
{
    //
    $cat=Categorie::find($id);
    return view('show',compact('cat'));
}

/**
 * Show the form for editing the specified resource.
 */
public function edit(string $id)
{
    //
    $cat=Categorie::find($id);
    return view('edit',compact('cat'));
}

/**
 * Update the specified resource in storage.
 */
public function update(Request $request, string $id)
{
    //
    $request->validate([
        'nom'=>'required|unique:categories|max:255',
    ]);
}

```

```
});
Categorie::update($request->all());
//rediriger l user à la vue index
return view('index')->with('success','catégorie bien modifiée !!!');

}

/**
 * Remove the specified resource from storage.
 */
public function destroy(string $id)
{
    //
    $cat =Categorie::find($id);
    $cat->delete();
    return view('index')->with('success','catégorie bien supprimée !!!');

}
}
```

Etape 6 : Définir les routes permettant d’accéder aux actions du contrôleur

Au niveau du fichier routes/ web.php

```
Route::resource('categories', CategorieController::class);
```

La Liste des routes générées par la dernière ligne :

URI	Verb http	Action	Route name
/ categories	Get	index	categories.index
/categories	Post	Store	categories.store
/categories/{id}	Get	Show	categories.show
/categories/{id}/edit	Get	Edit	categories.edit
/categories/{id}	PUT	Update	categories.update
/categories/{id}	Delete	Destroy	Categories.destroy

Travail à faire :

1. Installer l’environnement de travail
2. Refaire cette application de gestion de produit :
 - a. Créer les migrations en gérant la relation one to many
 - b. créer les models

- c. créer les factories et seeders
- d. créer les contrôleurs de ressource
- e. créer les routes
- f. créer les views : `index.blade.php` , `create.blade.php` ,
`show.blade.php` , `edit.blade.php`